

RANDOM FORESTS

IMPROVING VARIANCE

- Let $X_1, X_2, \ldots, X_n$ be i.i.d. random variable each with variance σ^2
- Define $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$
- What is the variance of $\overline{X}$?

$$Var(\overline{X}) = \frac{\sigma^2}{n}$$

- The using the mean of random variable will result in a lower variance

APPLYING THIS PRINCIPLE TO TREES

- Reduce the variance in the predictions by building a lot trees and averaging the results to produce a final prediction
- But how can we get more data to build new trees?

BOOTSTRAP AGGREGATION (ING)

- **Bagging** is an *ensemble** method which uses bootstrap sampling to generate multiple predictions and
  - Take a random sample of the training data *with replacement*
  - Build a tree using the sampled data
  - Make predictions with the model
  - Repeat B times
  - Average predictions across all models for final prediction

**An ensemble method an any method that involves aggregating predictions over multiple models*

MORE ABOUT BAGGING

- Bagging can be using for both regression and classification
  - Regression predictions are the mean
  - Classification predictions are the majority vote
- Bagging can also be used with models that aren't decision trees

WHAT IF THE PREDICTIONS ***AREN'T*** INDEPENDENT?

- The reduction in variance isn't as strong when variables are **correlated**

$$Var(X_1 + X_2) = Var(X_1) + Var(X_2) + 2Cov(X_1, X_2)$$

- The random forest method is similar to bagging except that it tries to de-correlate the trees through more randomization

ENSEMBLE VARIANCE: SETUP

Goal: how does the variance of the average depend on M and ρ?

- Let $T_1, \ldots, T_M$ be predictions from M tree predictors (for a fixed input x). Denote
  - $\mu = E[T_i]$ (assumed same for all trees),
  - $\sigma^2 = \mathrm{Var}(T_i)$ (same for all trees),
  - $\rho = \mathrm{Corr}(T_i, T_j)$ for $i \neq j$ (average pairwise correlation).
  - $\mathrm{Cov}(T_i, T_j) = \rho\sigma^2$ (since Corr = Cov/σ^2)

$$\mathrm{Var}(\bar{T}) = \mathrm{Var}\left(\frac{1}{M}\sum_{i=1}^{M} T_i\right) = \frac{1}{M^2}\mathrm{Var}\left(\sum_{i=1}^{M} T_i\right)$$

Expand the variance of the sum: "M variance terms + M(M−1) covariance terms"

$$\mathrm{Var}\left(\sum_{i=1}^{M} T_i\right) = M\sigma^2 + M(M-1)\rho\sigma^2$$

ENSEMBLE VARIANCE: THE RESULT

Start from the expansion, then divide by M^2:

$$\mathrm{Var}(\bar{T}) = \frac{M\sigma^2 + M(M-1)\rho\sigma^2}{M^2}$$

Simplify, then rearrange:

$$\mathrm{Var}(\bar{T}) = \frac{\sigma^2}{M} + \frac{M-1}{M}\rho\sigma^2$$

$$\boxed{\mathrm{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2}$$

What it tells us

- $\rho = 0$: Var = σ^2/M, matching slide 2
- $\rho = 1$: Var = σ^2, so averaging gives no benefit
- $M \to \infty$: Var $\to \rho\sigma^2$, so more trees cannot remove the correlated part
- **Random forests lower ρ by de-correlating the trees**

TWO SOURCES OF RANDOMNESS

- **1. Bootstrap sampling**
  - Each tree is trained on a **bootstrap sample** of the observations
  - Different trees see different data
- **2. Random feature selection**
  - At **every split** only consider a **random sample of m features** (from p total) as split candidates
  - Typical choice: $m \approx \sqrt{p}$
  - Finds alternative paths to decisions

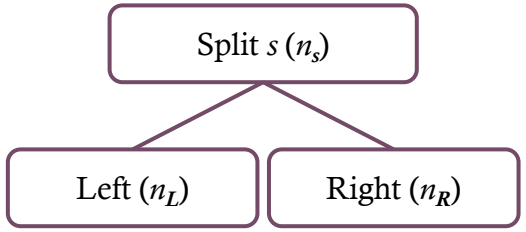
- With bagging alone, most trees pick the very strong predictor, so predictions stay similar. Random feature selection adds a second source of randomness that forces different splits, de-correlating the trees.

MODEL INTERPRETATION

- The bad news: The increase in predictive ability decreases the interpretability of the model (compared to one tree)
    - We can't just draw the tree anymore
- The good news: We can get an overall idea of *feature importance* measured by:
    - Finding splits using a particular feature
    - Computing the average impurity reduction across all trees
    - Features with the largest average decrease in impurity (Gini or entropy) are the most important
    - Scikit-learn scales so that feature importances sum to 1

MEAN DECREASE IN IMPURITY: CALCULATION

- **How it is calculated**
  - Using the impurity measure (Gini or entropy) from the Model Interpretation slide



1. One split *s* on feature *j*

$$\Delta i_s = i(\mathrm{s}) - \frac{n_L}{n_s} i(\mathrm{L}) - \frac{n_R}{n_s} i(\mathrm{R})$$

2. Weight each split by its share of samples, sum by feature, average over the M trees

$$\mathrm{MDI}_j = \frac{1}{M} \sum_{t=1}^{M} \sum_{s \in S_{t,j}} \frac{n_s}{n} \Delta i_s$$

Δi_s = impurity decrease at split s (weight n_s/n); $S_{t,j}$ = splits on feature j in tree t

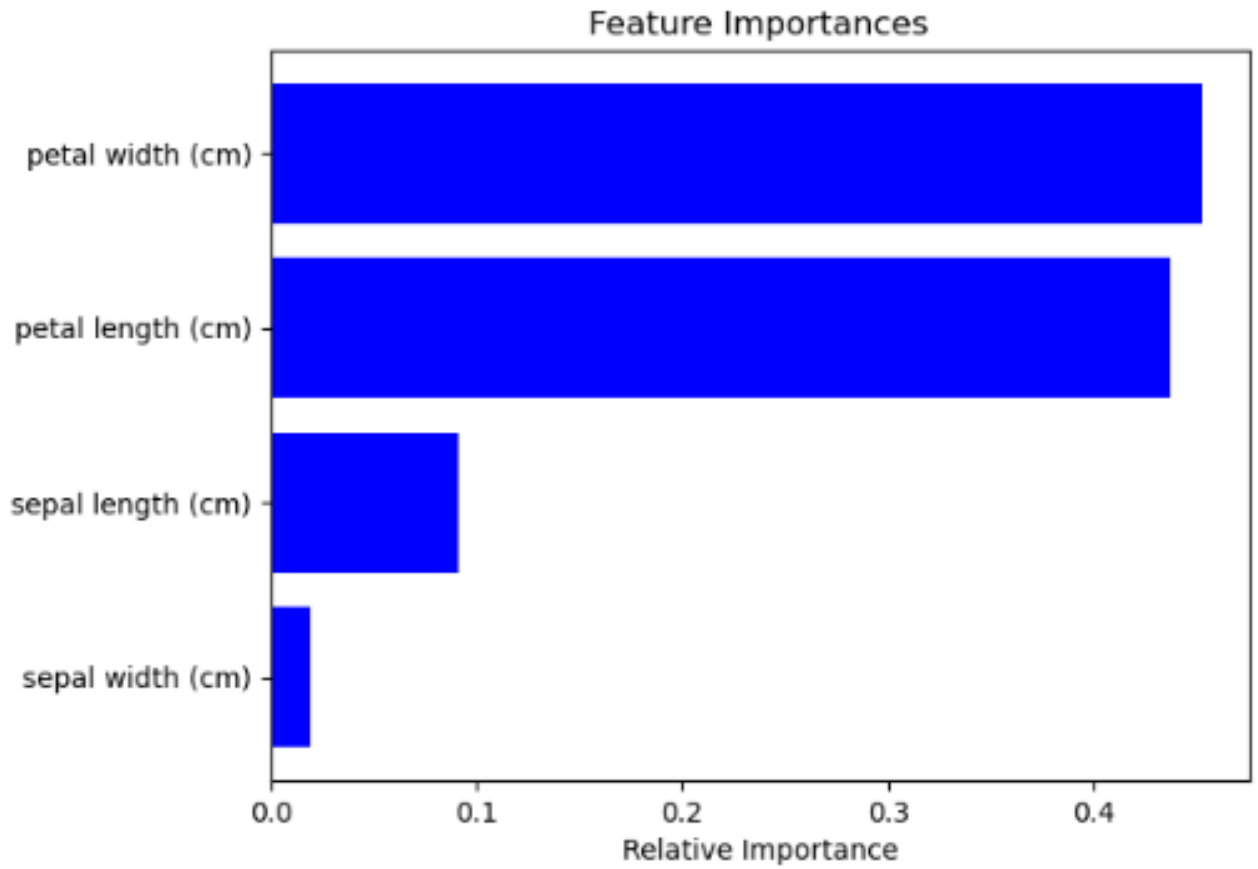
scikit-learn: importances are normalized to sum to 1

MEAN DECREASE IN IMPURITY: INTERPRETATION

- **How to interpret it**
  - Larger value: the fitted model used that feature more to reduce impurity
  - Not causal: it does not show the feature causes the outcome, or how changing it affects predictions
- **Caveats**
  - Correlated features can share or distort importance
  - Impurity-based importance can favor high-cardinality features
  - Values depend on the model and the dataset

Takeaway: MDI shows how much each feature helped reduce split impurity in this fitted forest.

EXAMPLE OF FEATURE IMPORTANCE



OUT OF BAG ERROR ESTIMATION

- In a bootstrap sample, on average $2/3$ of the instances are included
- The remaining $1/3$ of the instances are not used to fit a given bagged tree
  - The unused instances are called **out-of-bag** (OOB)
  - We can make predictions instance i using trees in which the ith was OOB
  - A single prediction for instance i is computed through aggregation the trees
  - The overall OOB error is asymptotically *equivalent* to performing leave-one-out cross-validation (if the number of trees is large enough)

OOB SAMPLES

- **Probability an instance is out-of-bag**
  - Each of the n draws (with replacement) picks a given instance with probability $1/n$
  - So one draw misses it with probability $1 - 1/n$
  - All n draws miss it, so P(OOB) = $(1 - 1/n)^n$
  - As $n \to \infty$: $(1 - 1/n)^n \to e^{-1} \approx 0.368$
  - So $\approx$ 36.8% of instances are OOB and $\approx$ 63.2% (about $2/3$) are in the bag

- **How OOB samples are used**
  - OOB error: predict each instance using only trees that did not see it; it approximates leave-one-out CV
  - Tuning: compare OOB error across M and m without cross-validation
  - Choosing M: add trees until the OOB error levels off
  - OOB permutation importance: shuffle a feature in the OOB samples and measure the rise in error

Each tree leaves out about $1/3$ of the data, so the forest gets a built-in validation set at no extra cost.

TUNING PARAMETERS (BAGGING AND RF)

- `n_estimators`: number of trees in forest
  - Increasing does not lead to overfitting
- `criterion`: Gini impurity or entropy
- `max_features`: Number of features to consider at each split
- Tuning parameters associated with base estimator